

ASTRUM DRIVE

ASTRA

Verifiable AI for Scientific Discovery

A multi-model research platform that turns scientific objectives into falsifiable conjectures, reviewed executable tests, reproducible evidence, and the next decision.

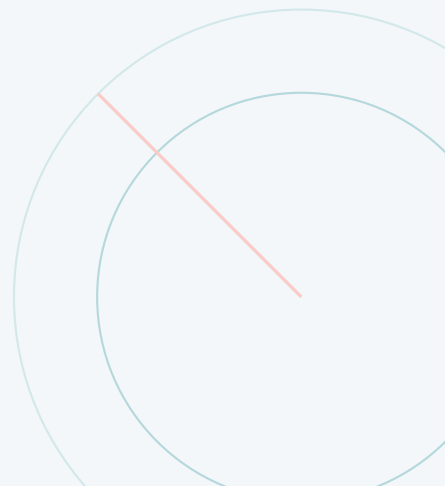
Investor, VC, Client and Strategic Partner Brief

Executive narrative, product value, customer applications, system architecture, algorithmic workflow, validation model, commercialization roadmap and risks.

Nelson Bolívar

Founder and Lead Architect | Astrum Drive

July 2026 | Discussion document



Executive thesis

ASTRA makes AI-generated scientific reasoning inspectable.

Most AI systems optimize the answer. ASTRA optimizes the evidence chain: independent models propose and challenge a claim, a specialist writes a falsifiable validator, another model audits that validator, computation engines execute it, and a human controls what becomes accepted knowledge.

The problem

Scientific and engineering teams can generate hypotheses and code faster with AI, but the bottleneck moves downstream: deciding whether the produced result actually tests the intended claim. Plausible prose, a successful program, or a printed PASS is not sufficient evidence.

This creates four practical risks:

- ▶ hidden changes in assumptions, domains or boundary conditions;
- ▶ validators that confirm their own construction;
- ▶ numerical checks presented as universal proofs;
- ▶ fragmented evidence across chats, notebooks and tools.

The ASTRA response

ASTRA coordinates complementary AI models and deterministic scientific engines around one shared objective. Generation, code authorship, code review, execution, result analysis and research navigation are deliberately separated.

The product is designed to produce:

- ▶ a precise and falsifiable statement;
- ▶ executable validation code with explicit failure paths;
- ▶ raw output from symbolic, numerical or logical engines;
- ▶ a conservative interpretation tied to the original objective;
- ▶ a persistent record that experts can reproduce and challenge.

Investment proposition. ASTRA is an orchestration and assurance layer for AI-assisted R&D. It is model-flexible, engine-extensible and deployable close to private data and scientific software. The near-term opportunity is to convert an operational research system into a pilot-ready product for teams that need faster experimentation without surrendering traceability.

Category reference. Google DeepMind's Aletheia research agent demonstrates the scientific value of a generator-verifier-reviser loop powered by Gemini Deep Think. ASTRA shares that epistemic pattern, but implements it as a multi-model, execution-grounded and local-first product architecture for scientific R&D.

What exists today

3

specialized model roles in the production configuration

8

explicit stages from shared objective to human approval

1

persistent evidence chain across every cycle

The current system includes a browser application, an MCP interface for agent control, local and remote execution, detached long-running jobs, benchmark cases, report persistence, optional formal proof and CAS engines, package extensions, retry and autofix logic, and human approval gates.

Part I – Product and commercial view

One product, three audiences

Audience	Immediate value	Typical entry point
R&D teams	Move from a research question to a reproducible computational test while preserving assumptions and failed paths.	Theoretical physics, symbolic models, differential equations, optimization, simulation validation.
Engineering clients	Audit AI-generated scientific code and compare results across independent engines or internal packages.	Model verification, solver regression, design-space exploration, mathematical QA.
Investors / partners	Back an assurance layer positioned between frontier models and high-value scientific workflows.	Design partnerships, vertical modules, enterprise deployment, compute and model integrations.

The customer job

When a technical team has a consequential scientific claim, **it needs** to translate that claim into a test that can fail, execute it with the right tools, and preserve enough evidence for another expert to reproduce the decision. **ASTRA packages that workflow as a governed research cycle.**

Candidate use cases

- ▶ symbolic identities, invariants and limiting cases;
- ▶ general relativity metrics and curvature calculations;
- ▶ quantum operators, states and consistency checks;
- ▶ differential equations and boundary-value problems;
- ▶ fluid mechanics and dimensional validation;
- ▶ counterexample search with Z3;
- ▶ formal logic and Lean 4 proofs;
- ▶ optimization using internal scientific packages;
- ▶ cross-validation through Mathematica;
- ▶ reproducibility audits for model-generated code.

Value creation

Faster path to evidence

Automates the transitions between hypothesis, formalization, executable test, correction and interpretation.

Lower verification friction

Places a code review and deterministic execution between model output and scientific acceptance.

Reusable organizational knowledge

Stores validated results, refutations, scripts and reports instead of losing them in isolated conversations.

Flexible scientific stack

Routes work through local, remote, open-source, formal and in-house computation engines.

Commercial models to validate

The architecture supports several commercial forms; these remain go-to-market hypotheses rather than declared pricing:

- ▶ individual and team subscriptions for research orchestration;
- ▶ enterprise licensing for private deployment and governed evidence retention;
- ▶ paid vertical modules for specific scientific domains;
- ▶ integration, validation and joint research programs with internal software, compute, model and scientific-software partners.

Why ASTRA can be differentiated

	Conventional AI assistant	ASTRA
Primary output	A response or code artifact	A traceable evidence chain
Model structure	Often one model and one context	Multiple models with separated responsibilities
Validation	Optional, user-initiated	Mandatory execution in the core cycle
Code quality control	Same model may author and judge	Independent reviewer can reject and request revision
Scientific engines	Ad hoc tool use	Routed oracle with explicit engine metadata
Failure semantics	Errors may be folded into prose	REFUTED, CODE_ERROR, API_ERROR, weak evidence
Knowledge governance	Chat history	Reports, branches, axiomatic base and human gate
Deployment posture	Primarily hosted	Local-first with optional remote execution

Defensibility thesis

Workflow IP and calibration. The valuable asset is not a single prompt. It is the evolving set of role contracts, review rules, verdict guards, correction behavior, benchmark cases and domain-specific validation patterns.

Engine and package graph. ASTRA can accumulate verified adapters and test protocols across symbolic algebra, SMT, numerical science, formal proof, Mathematica and proprietary packages.

Evidence and failure data. With appropriate customer consent and governance, recurring validator defects, refutation patterns and correction outcomes can improve system calibration.

Model independence. The orchestration layer can adopt stronger models without rebuilding the scientific workflow. Models are replaceable capabilities; the evidence architecture persists.

Reference comparison: ASTRA and Aletheia / Gemini Deep Think

Google DeepMind describes Aletheia as a mathematics research agent powered by an advanced Gemini Deep Think model. Its documented loop generates a candidate solution, uses a natural-language verifier, revises minor defects, restarts critically flawed solutions and can admit failure. It also uses search, web browsing and computational tools for long-horizon mathematical research.¹ The associated research paper reports evaluations from Olympiad and PhD-level exercises through open research problems.²

Dimension	Aletheia / Gemini Deep Think	ASTRA
Foundation	An advanced Gemini Deep Think reasoning model wrapped in a mathematics research agent.	A role-separated ensemble: Codex, Antigravity and Claude, coordinated around one objective.
Core loop	Generator, natural-language verifier and reviser; critical failures restart generation.	Parallel conjecture, cross-critique, synthesis, code author, independent code reviewer, oracle, analyst and navigator.
Focus	Long-horizon natural-language proof and professional mathematics research.	Executable scientific validation across symbolic, numerical, logical and package-specific workflows.
Verification	Natural-language verification, tools, code-assisted checks and human expert grading.	Pre-execution code audit plus deterministic engine output, verdict guards, result analysis and human approval.
Research search	Google Search and web browsing are documented parts of the research agent.	Core governance currently centers on execution; literature navigation can be supplied by an upstream agent and remains a productization target.
Tool ecosystem	Intensive tool use within Google's research system.	Routed local or remote oracle: Python science stack, Z3, Lean, CAS tools, Mathematica bridge and internal packages.
Deployment	Google-hosted frontier research capability.	Local-first orchestration with optional remote compute and agent access through MCP.
Memory	Iterative solution state and research context.	Shared objective, axiomatic base, explicit refutations, branch memory, reports and human admission gate.
Evidence maturity	Expert-graded benchmark and research results reported by Google DeepMind and collaborators.	Operational integration cycles and regression tests; external domain benchmarks and customer pilots are the next evidence threshold.

What is genuinely shared

- ▶ verification is a process, not a final prompt;
- ▶ candidate work must be revised or discarded;
- ▶ the system should be able to admit failure;
- ▶ proof and refutation should be pursued together;
- ▶ tools and code can strengthen reasoning;
- ▶ expert review remains part of the evidence chain.

¹Google DeepMind, "Accelerating Mathematical and Scientific Discovery with Gemini Deep Think," February 11, 2026.

²Feng et al., "Towards Autonomous Mathematics Research," arXiv:2602.10177, 2026.

Where ASTRA makes a different product bet

Aletheia is the strongest public reference for the research-agent pattern; ASTRA is not presented as matching its published mathematical performance. ASTRA's product bet is that customers will also need **model plurality, validator-code review, engine-grounded evidence, private deployment, internal-package integration and organizational approval workflows**. The opportunity is to turn the pattern demonstrated at the frontier into an auditable R&D operating layer.

Important distinction. Gemini Deep Think is a specialized reasoning mode built on Gemini 3.1 Pro.^a Aletheia is the agentic research system built around that capability. ASTRA is a separate orchestration architecture and does not use Aletheia or Deep Think. Its current Antigravity role uses a high-reasoning Gemini 3.1 Pro model alongside Codex and Claude, with different responsibilities and independent execution.

^aGoogle DeepMind, "Gemini 3.1 Deep Think," accessed July 2026.

Strategic interpretation

For investors and clients, DeepMind's work reduces category risk: it demonstrates that iterative generation, verification, revision, balanced proof/refutation and human collaboration can materially extend model usefulness in research. ASTRA's remaining commercial question is whether its broader, deployable assurance layer can create measurable value in customer workflows outside a frontier research laboratory.

Current proof of operation

On July 23, 2026, the local production configuration completed an end-to-end integration cycle:

- ▶ Claude Opus 4.8 produced a proposed validator.
- ▶ GPT-5.6 Sol rejected the first artifact as non-executable and epistemically invalid.
- ▶ Claude revised it; GPT-5.6 Sol approved the corrected validator.
- ▶ Exact polynomial normalization and Z3 returned a decisive PASS.
- ▶ GPT-5.6 Sol independently interpreted the code and evidence as VALIDATED.
- ▶ Antigravity navigation marked the integration objective resolved and proposed boundary tests.

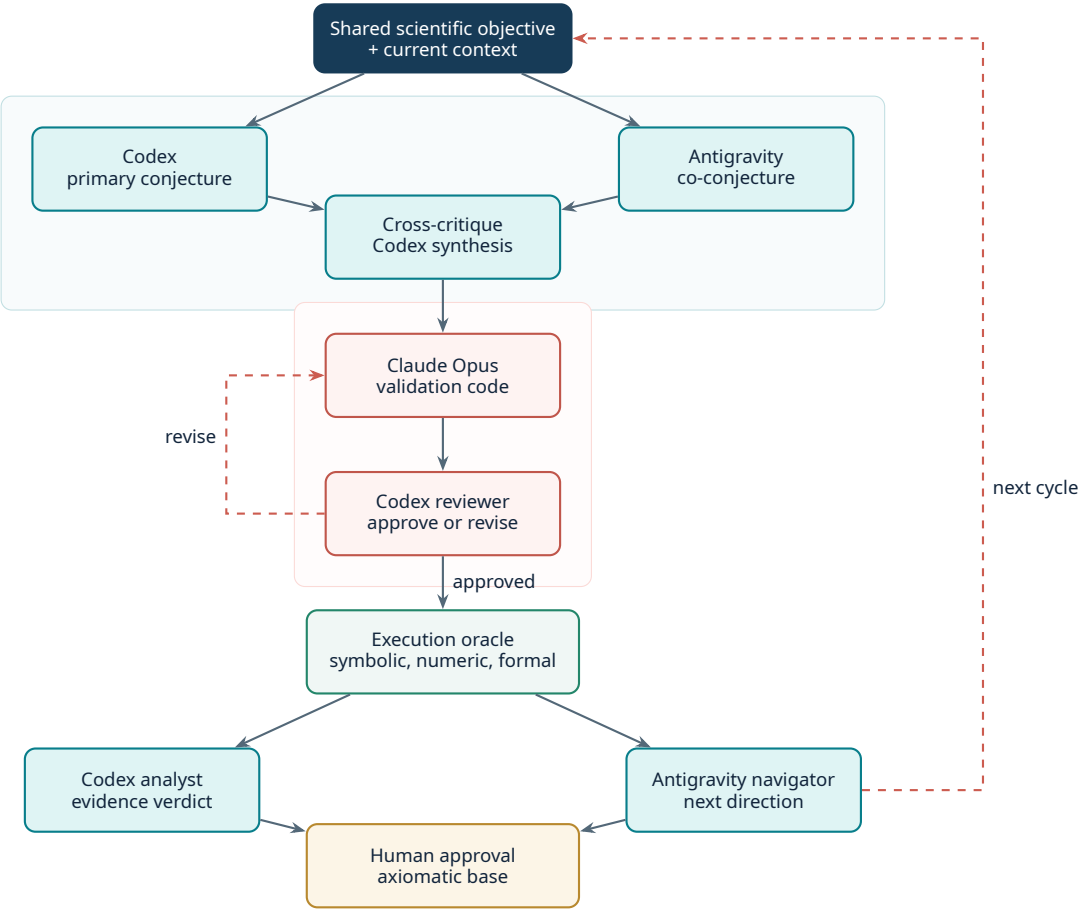
What this proves: the implemented models, review loop, execution oracle and navigator can exchange artifacts and close a complete cycle. **What it does not prove:** market adoption, universal scientific correctness, or readiness for every enterprise environment. Those are the next validation programs.

Metrics for design-partner pilots

Product metric	Why it matters
Time from objective to first reproducible test	Measures research-cycle compression
Independent reviewer rejection rate	Reveals how often first-pass validators are unsafe
Revision success rate	Measures value of the cross-model correction loop
False-acceptance rate on adversarial benchmarks	Core assurance and trust metric
Reproduction rate on a second machine or engine	Tests portability of evidence
Human review time per accepted result	Measures whether traceability reduces expert burden

Part II – System and algorithmic detail

Production architecture



Role separation is the central control. Codex leads conjecture, synthesis, code review and evidence analysis. Antigravity contributes an independent conjecture and maintains research direction. Claude authors and revises validation code. Deterministic engines, not model consensus, produce the evidence.

Interfaces and deployment

Layer	Current implementation
Interaction	Browser interface, command-line subprocess API and MCP tools for agents
Orchestration	Asynchronous Python pipeline with progress, timeouts, fallbacks and detached jobs
Models	Subscription-authenticated Codex, Claude Code and Antigravity CLIs; optional API providers
Execution	Local Python and optional remote Linux worker; engine-specific routing
Persistence	Reports, cycle state, research branches, benchmark results and human approval
Security posture	Credentials remain in local configuration; private execution is supported; enterprise controls remain a hardening priority

The core algorithm

Inputs: shared objective G , established context K , current direction D , maximum code revisions R , execution policy O .

```

1.  $p_c \leftarrow \text{Codex.propose}(G, K, D)$ 
2.  $p_a \leftarrow \text{Antigravity.propose}(G, K, D)$ 
3.  $q_c \leftarrow \text{Codex.critique}(p_a); \quad q_a \leftarrow \text{Antigravity.critique}(p_c)$ 
4.  $h \leftarrow \text{Codex.synthesize}(G, p_c, p_a, q_c, q_a)$ 
5.  $v_0 \leftarrow \text{Claude.translate}(G, h)$ 
6. for  $i = 0, \dots, R$ :
     $r_i \leftarrow \text{Codex.review}(G, h, v_i)$ 
    if  $r_i = \text{APPROVED}$ : break
     $v_{i+1} \leftarrow \text{Claude.revise}(v_i, r_i)$ 
7.  $e \leftarrow \text{Oracle.execute}(v_i, O)$ 
8.  $s \leftarrow \text{Guard}(v_i, e)$ 
9.  $a \leftarrow \text{Codex.analyze}(G, h, v_i, e, s)$ 
10.  $n \leftarrow \text{Antigravity.navigate}(G, K, h, a)$ 
11.  $\text{HumanGate}(h, v_i, e, a, n)$ 

```

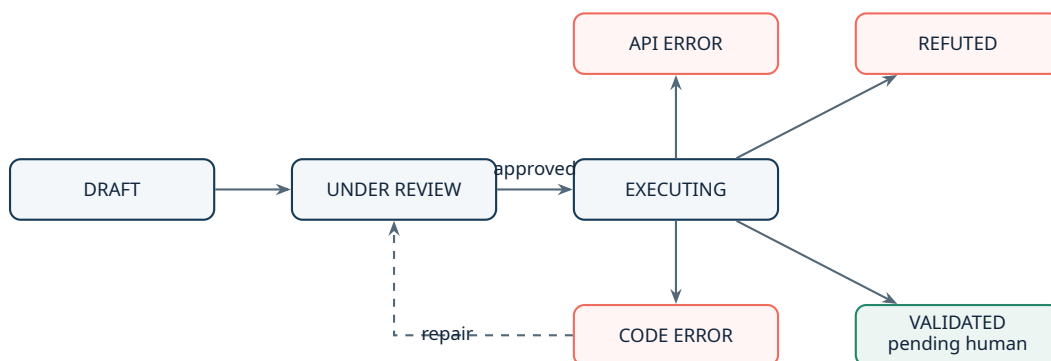
[parallel]

Conservative decision semantics

The orchestration enforces several monotonic constraints:

- ▶ a crashed validator cannot become VALIDATED;
- ▶ an explicit FAIL cannot be promoted by explanatory prose;
- ▶ a printed PASS is evidence, not authority;
- ▶ missing optional engines fail explicitly;
- ▶ a reviewer can reject circular, unreachable or sampling-only proof logic;
- ▶ final admission to the axiomatic base requires a human decision.

State model



Interpretation boundary. In ASTRA, VALIDATED means supported by the recorded validator, execution and assumptions. It does not mean absolute truth. The evidence remains open to stronger proof, independent replication and expert review.

Scientific engine and extension model

Oracle portfolio

Engine class	Examples	Validation role
Symbolic	SymPy, SageMath, Maxima	Exact simplification, identities, derivatives, algebraic residuals
Logic / proof	Z3, Lean 4 + Mathlib	Counterexample search, satisfiability, kernel-checked theorems
Numerical	NumPy, SciPy, mpmath, QuTiP	ODEs, high-precision checks, quantum systems, sweeps
Domain pack-ages	EinsteinPy, fluids, Pint	Curvature, fluid models, units and physical consistency
Independent CAS	Mathematica, bridge, Cadabra	Cross-engine validation, tensor algebra, notebook workflows
Internal soft-ware	GR_python, pyWarp-Factory, warp-bubble optimization	Reuse of proprietary or project-specific research capabilities

Extension contract

Any scientific package can become part of the oracle when a validator:

1. declares reproducible inputs, versions and assumptions;
2. imports the package in the selected environment;
3. computes a quantity that discriminates the claim from its negation;
4. exposes exact output, errors and tolerances;
5. includes reachable failure behavior;
6. preserves the script and result for review.

Agent control through MCP

ASTRA exposes operations for direct execution, complete cycles, progress probes, detached jobs, job polling and remote-worker status. A conversational agent can therefore plan a test and invoke ASTRA without manually copying code, while ASTRA retains the evidence and decision semantics.

Illustrative client instruction:

"Formulate two competing explanations for this curvature result. Use ASTRA to construct a validator with an exact symbolic leg and an independent numerical leg. Do not accept a printed PASS until the code, residuals and assumptions have been reviewed."

Enterprise productization priorities

- ▶ workspace authentication and role-based access;
- ▶ signed evidence bundles and immutable audit metadata;
- ▶ dependency and container pinning;
- ▶ job isolation and compute quotas;
- ▶ private model and on-premise connectors;
- ▶ team review queues and approval policy;
- ▶ observability, cost and latency dashboards;
- ▶ benchmark governance and release qualification.

Commercialization roadmap

Phase	Product objective	Evidence required before advancing
Now	Operational multi-model research loop and scientific oracle	Repeated benchmark passes; documented end-to-end cycles; reproducible local installation
Pilot hardening	Deploy with a small number of design partners	Measured time-to-evidence, reviewer correction value, reproduction rate and expert satisfaction
Team product	Shared projects, governed approvals and scalable jobs	Repeat use, retained teams, security acceptance and clear economic buyer
Vertical modules	Domain-specific validators, packages and benchmark suites	Superior outcomes in selected high-value workflows
Platform expansion	Private deployment, partner engines and model marketplace	Reliable operations, integration demand and sustainable unit economics

Near-term capital and partnership priorities

1. **Product engineering:** packaging, secure execution, observability, collaboration and evidence management.
2. **Scientific calibration:** adversarial benchmarks, false-acceptance measurement and expert-reviewed domain suites.
3. **Design partnerships:** two or three workflows with high verification cost and measurable research-cycle value.
4. **Distribution:** model, compute, scientific-software and research institution integrations.

Risks and mitigations

Risk	Why it matters	Mitigation direction
Validator mismatch	Code may test a weaker or different claim	Independent code review, human gate, benchmarked failure patterns
Model latency and quota	Maximum-quality cycles can take minutes	Phase budgets, fallbacks, caching, fast and deep profiles
False confidence	Users may overread VALIDATED	Explicit semantics, evidence display, independent engines, approval policy
Scientific breadth	No system is expert in every domain	Vertical focus, package adapters, domain reviewers and scoped claims
Enterprise security	Generated code executes with real resources	Isolation, allowlists, containers, credentials policy and audit logs
Go-to-market diffusion	"Science" is too broad as an initial segment	Start with measurable design-partner workflows and a specific buyer

The next proof points

- ▶ demonstrate lower false-acceptance rates than a single-model baseline;
- ▶ reproduce partner results across a second environment or engine;
- ▶ quantify expert time saved per accepted or refuted claim;
- ▶ identify one vertical where ASTRA becomes a paid part of the weekly R&D workflow.

Engagement proposal

The product thesis in one sentence

ASTRA is the verification and coordination layer that allows organizations to use frontier AI in scientific work while preserving falsifiability, reproducibility and human control.

For a prospective client

Select one consequential workflow with:

- ▶ a clear scientific decision and current baseline process;
- ▶ existing code, equations, simulations or internal packages;
- ▶ measurable verification effort or rework;
- ▶ an expert who can judge whether ASTRA preserves the real claim.

A pilot should compare baseline and ASTRA-assisted work on time-to-evidence, reproducibility, review effort and false acceptance.

For an investor or VC

The immediate diligence questions are concrete:

1. Does independent review materially improve generated validators?
2. Which customer segment has the highest cost of unverifiable AI output?
3. Can the evidence architecture become a repeatable team workflow?
4. Which data, adapters and calibration assets compound with usage?
5. Can private deployment and scientific specialization support durable economics?

For a strategic partner

ASTRA can provide a governed research workflow around:

- ▶ frontier models that need credible scientific demonstrations;
- ▶ compute platforms that want traceable high-value workloads;
- ▶ scientific-software vendors seeking agent-native interfaces;
- ▶ research organizations building reproducible AI-assisted methods.

Proposed next step. Define a 4–8 week design-partner study around one scientific workflow, one baseline, one reproducibility target and an agreed set of acceptance metrics. The goal is not a demo; it is evidence that ASTRA changes a real R&D decision.

ASTRA

From plausible answers to reviewable evidence.

Nelson Bolívar | Founder and Lead Architect | Astrum Drive
Investor, client and strategic partner discussions | July 2026